
Аннотация

Реализация в отладчике GDB поддержки вызова функций с векторным ABI для архитектуры RISC-V

Радькин Кирилл Алексеевич

В рамках данной выпускной квалификационной работы была реализована поддержка вызова функций с векторным ABI для архитектуры RISC-V в отладчике GDB. В работе рассмотрены особенности архитектуры RISC-V и векторного расширения данной архитектуры, процесс генерации отладочной информации компиляторами GCC и LLVM для архитектуры RISC-V. Для выполнения поставленной цели в отладчике GDB была реализована возможность взаимодействия с векторными типами и соглашение о вызовах функций для векторных типов RISC-V. Корректность работы реализованных возможностей была проверена в рамках тестовой инфраструктуры GDB с использованием специально написанных для этого тестовых сценариев.

Содержание

1	Введение	3
2	Постановка задачи	5
3	Обзор архитектуры RISC-V	6
3.1	Общая информация	6
3.2	Векторное расширение	7
3.3	Векторные типы	9
3.4	Соглашение о вызовах	10
4	Отладчик GDB	12
4.1	Основная информация и возможности	12
4.2	Отладочная информация: DWARF	12
4.3	Необходимая информация о внутреннем устройстве GDB . .	15
5	Описание практической работы	19
5.1	Исправление генерации отладочной информации в компиляторе LLVM	19
5.2	Реализация в GDB	20
6	Заключение	26

1 Введение

В последние годы архитектура RISC-V привлекает всё больше внимания в академической и промышленной среде благодаря своей открытости и модульности. Модульность является одной из ключевых особенностей архитектуры RISC-V. Архитектура состоит из базового набора целочисленных инструкций, который должен присутствовать в любой реализации, и из дополнительных расширений, которые можно включать по необходимости. Одним из них является векторное расширение (RISC-V Vector Extension, RVV), предназначенное для эффективной параллельной обработки данных.

Разработка и отладка программного обеспечения для RISC-V с поддержкой RVV требует соответствующих инструментов. Одним из таких инструментов является отладчик.

Отладчик GDB (GNU Debugger) предоставляет пользователю множество возможностей для ускорения и упрощения процесса отладки. Однако на момент начала работы GDB имел ограниченную поддержку RVV, в частности отсутствовала поддержка соглашения о вызовах для векторных типов, которое необходимо для работы механизма вызова произвольных функций с помощью GDB, что, в свою очередь, затрудняло возможности для анализа и отладки подобных функций.

Цель данной работы – расширить функциональность GDB за счёт реализации поддержки вызова функций с использованием векторного ABI для архитектуры RISC-V. Это позволит разработчикам вызывать такие функции вручную из GDB, корректно передавая аргументы в векторные регистры, получать и интерпретировать возвращаемые значения в соответствии с соглашением о вызовах для векторных типов.

В процессе работы был проведён анализ соглашения о вызовах, рассмотрена структура внутренних компонентов GDB, отвечающих за передачу аргументов при вызове функций, а также реализованы необходимые изменения для корректной обработки векторных типов. Разработанная реализация была интегрирована в исходный код GDB и протестирована на наборе функций с различными комбинациями векторных типов в качестве аргументов, что позволило убедиться в корректности ее работы.

Практическая значимость данной работы заключается в повышении

уровня поддержки GDB для архитектуры RISC-V, что способствует более эффективной разработке и отладке программного обеспечения для данной архитектуры.

2 Постановка задачи

Цель работы: реализовать в отладчике GDB для архитектуры RISC-V поддержку вызова функций, использующих векторный ABI.

Для выполнения поставленной цели были сформулированы следующие задачи:

1. Изучить генерацию отладочной информации для RISC-V компиляторами GCC и LLVM
2. Реализовать RISC-V Vector Calling Convention в отладчике GDB
3. Поддержать определение векторных типов и работу с ними в GDB

3 Обзор архитектуры RISC-V

3.1 Общая информация

RISC-V – архитектура команд (ISA), основанная на концепции RISC, была разработана в 2010 году в Университете Калифорнии в Беркли. Её задачей было обеспечение как учебного процесса, так и практического использования в индустрии.

Открытость это одна из ключевых особенностей данной архитектуры. Спецификации RISC-V находятся в открытом доступе. Архитектура свободна от лицензионных отчислений, что позволяет использовать ее без юридических или финансовых ограничений.

Еще одним важным свойством архитектуры является ее модульность. Архитектура включает обязательный базовый набор целочисленных инструкций, который должен реализовываться во всех системах, а также предоставляет ряд дополнительных расширений, подключаемых по мере необходимости. Существует 4 варианта базового набора целочисленных инструкций: RV32I и RV64I, предназначенные для 32- и 64-битных адресных пространств (в терминах спецификации RISC-V это обозначается как XLEN), пространства, и RV32E и RV64E, облегченные версии которые были добавлены для поддержки маломощных микроконтроллеров и обладающие уменьшенным количеством целочисленных регистров. Дополнительные расширения можно разделить на две группы: стандартные и пользовательские. Среди стандартных дополнительных расширений можно выделить следующие:

- М – добавляет операции целочисленного умножения и деления
- А – добавляет атомарные операции с памятью
- F – добавляет регистры для значений с плавающей точкой и соответствующие операции
- D – добавляет регистры для значений с плавающей точкой двойной точности
- C – набор сжатых инструкций (с уменьшенной длиной)
- V – векторные регистры и соответствующие операции

- **B** – побитовые операции
- **Zicsr** – инструкции для работы с регистрами состояния и статуса ядра (Control and Status Registers, CSR)
- **Zifencei** – инструкции для синхронизации потока инструкций и потока данных

Кроме того, существует аббревиатура **G** для обозначения базового набора расширений **IMAFD_Zicsr_Zifencei**.

Архитектура поддерживается организацией RISC-V International, которая разрабатывает и утверждает стандартные расширения.

3.2 Векторное расширение

Векторное расширение (RISC-V Vector Extension, RVV) описывает набор инструкций и регистров, предназначенных для эффективной параллельной обработки данных.

Каждое ядро, поддерживающее данное расширение, определяет два значения:

1. **ELEN** – максимальный размер векторного элемента (в битах), который может быть использован или создан векторной операцией. $ELEN \geq 8$ и должно быть степенью 2.
2. **VLEN** – количество бит в одном векторном регистре. $2^{16} \geq VLEN \geq ELEN$, и должно быть степенью 2.

Векторное расширение добавляет 32 векторных регистра (**v0-v31**) и 7 непривилегированных CSR (**vstart**, **vxstat**, **vxrm**, **vcsr**, **vtype**, **vl**, **vlenb**).

- Регистр **vtype** является read-only CSR (т. е. регистр, доступный только для чтения) и может быть обновлен только с помощью специальных инструкций **vset{i}vl{i}**.

Данный регистр хранит в себе следующую информацию:

- Интерпретацию значений векторных регистров
- Способ группировки векторных регистров

- Способ обработки маскированных элементов
- Способ обработки элементов, длина которых превышает текущую длину вектора



Рис. 1: Поля регистра vtype

- **vill** – флаг, указывающий на нелегальное значение
- **vma** – если 0, то немаскированные элементы сохраняют свои значения после операций, если 1, то они могут как сохранить, так и быть переписаны единицами
- **vta** – аналогично **vma**, только для элементов, которые находятся за текущей длиной вектора
- **vsew** – текущее значение размера одного элемента (Selected Element Width, SEW)

vsew[2:0]			SEW
0	0	0	8
0	0	1	16
0	1	0	32
0	1	1	64
1	X	X	Reserved

Таблица 1: Соответствие между vsew[2:0] и SEW

Значение SEW – это размер в битах одного скалярного значения внутри векторного регистра. Соответственно, количество элементов в одном векторном регистре вычисляется как $VLEN/SEW$. В таблице 2 приведен пример для $VLEN = 128$ бит.

SEW	Количество элементов
8	16
16	8
32	4
64	2

Таблица 2: Пример для $VLEN = 128$ бит

- `vlmul` – текущее значение размера группы векторных регистров (Length Multiplier, LMUL)

Векторные регистры могут быть сгруппированы, чтобы одна векторная инструкция могла оперировать сразу с несколькими векторными регистрами. Это повышает эффективность обработки данных, позволяя работать сразу с несколькими регистрами (т. е. с векторами увеличенной длины). Значение LMUL представляет собой количество векторных регистров, сгруппированных вместе.

Кроме того, LMUL может быть дробным значением. Дробное LMUL означает, что используется только часть бит векторного регистра.

Таким образом, мы можем вычислить максимальное количество скалярных элементов, которыми может оперировать одна векторная инструкция с текущими SEW и LMUL, как $VLMAX = LMUL * VLEN / SEW$.

- Регистр `vl` является read-only CSR и может быть обновлен только с помощью специальных инструкций `vset{i}vl{i}`.

Данный регистр хранит в себе беззнаковое целочисленное значение, определяющее, с каким количеством скалярных элементов будет взаимодействовать векторная инструкция.

- Регистр `vlenb` является read-only CSR, который хранит в себе длину одного векторного регистра в байтах, т. е. $VLEN/8$.

3.3 Векторные типы

Векторными типами в данной работе называются типы данных, предоставляемые в рамках интерфейса на языке C для работы с векторным расширением. Данный интерфейс позволяет программисту написать код с использованием intrinsic-функций, которые преобразуются компилятором в определенные инструкции или небольшие наборы инструкций.

Существует несколько категорий векторных типов: целочисленные, с плавающей точкой, тип векторной маски (`vbool`), кортежные типы.

В названии каждого векторного типа закодированы параметры LMUL и SEW. Например, для типа `vint32m1_t` значение LMUL равно 1, значение SEW равно 32. Также, существуют конфигурации с дробным значением

LMUL. Например, для типа `vint8mf4_t` значение LMUL будет дробным и равным $1/4$.

Type	LMUL						
	1/8	1/4	1/2	1	2	4	8
<code>int_8</code>	<code>vint8mf8_t</code>	<code>vint8mf4_t</code>	<code>vint8mf2_t</code>	<code>vint8m1_t</code>	<code>vint8m2_t</code>	<code>vint8m4_t</code>	<code>vint8m8_t</code>
<code>int_16</code>	-	<code>vint16mf4_t</code>	<code>vint16mf2_t</code>	<code>vint16m1_t</code>	<code>vint16m2_t</code>	<code>vint16m4_t</code>	<code>vint16m8_t</code>
<code>int_32</code>	-	-	<code>vint32mf2_t</code>	<code>vint32m1_t</code>	<code>vint32m2_t</code>	<code>vint32m4_t</code>	<code>vint32m8_t</code>
<code>int_64</code>	-	-	-	<code>vint64m1_t</code>	<code>vint64m2_t</code>	<code>vint64m4_t</code>	<code>vint64m8_t</code>
<code>uint_8</code>	<code>vuint8mf8_t</code>	<code>vuint8mf4_t</code>	<code>vuint8mf2_t</code>	<code>vuint8m1_t</code>	<code>vuint8m2_t</code>	<code>vuint8m4_t</code>	<code>vuint8m8_t</code>
<code>uint_16</code>	-	<code>vuint16mf4_t</code>	<code>vuint16mf2_t</code>	<code>vuint16m1_t</code>	<code>vuint16m2_t</code>	<code>vuint16m4_t</code>	<code>vuint16m8_t</code>
<code>uint_32</code>	-	-	<code>vuint32mf2_t</code>	<code>vuint32m1_t</code>	<code>vuint32m2_t</code>	<code>vuint32m4_t</code>	<code>vuint32m8_t</code>
<code>uint_64</code>	-	-	-	<code>vuint64m1_t</code>	<code>vuint64m2_t</code>	<code>vuint64m4_t</code>	<code>vuint64m8_t</code>

Таблица 3: Целочисленные векторные типы

Для кортежных типов появляется дополнительный параметр `NFIELD` – число элементов в кортеже. Например, кортежный тип `vint32m2x3_t` объединяет 3 элемента с типом `vint32m2_t` ($LMUL = 2$, $SEW = 32$).

Для типов векторной маски схема кодировки отличается. Например, в названии типа `vbool8_t`, число 8 – это отношение $SEW/LMUL$. Соответственно, только к операциям с типами с равным отношением $SEW/LMUL$ может быть применена маска данного типа.

3.4 Соглашение о вызовах

Соглашение о вызовах (Calling Convention) – набор правил, являющихся частью ABI (Application Binary Interface), и определяющих особенности вызова функций. Этими правилами определяются способы передачи аргументов функции и получения результата ее работы (через регистры процессора и/или через стек, порядок размещения аргументов), а также способы вызова и возврата из функций. Далее в этом разделе я рассмотрю соглашение о вызовах для функций с векторными аргументами и возвращаемыми значениями.

Для передачи векторных аргументов в функции существует несколько правил. Одно из таких правил говорит, какие регистры должны использоваться для передачи аргументов и возвращаемого значения, а какие регистры (точнее, их значения) должны быть сохранены между вызовами. Регистр `v0` используется для передачи аргументов с типом векторной маски (т. е. различные типы `vbool`) и для возвращаемого значения соответствующего типа. Регистры `v8-v23` используются для передачи обычных векторных аргументов, кортежных векторных типов и оставшихся аргументов с

типом векторной маски. Эти же регистры используются для возвращаемого функцией значения с одним из вышеуказанных типов.

Также, должно быть обеспечено сохранение значений регистров `v1-v7` и `v24-v31`.

И последнее в списке, но не по важности, правило, по которому выбираются векторные регистры для расположения на них аргументов. В первую очередь, мы должны выбрать некоторый последовательный набор регистров, на котором может поместиться данный аргумент. Например, для типа `vint64m2_t` это может быть `v8-v9`, но не может быть `v8` и `v10`. Вторым пунктом в данном правиле говорится о том, что номер первого регистра в данном наборе должен быть кратен значению `LMUL`. Например, для типа `vint64m4_t` это могут быть наборы `v8-v11` или `v12-v15`, но не `v9-v12`.

Следует подчеркнуть, что поиск подходящего набора регистров каждый раз начинается с `v8`, следовательно, существует возможность, что номер первого векторного регистра из набора, присвоенного векторному аргументу может быть меньше номера первого векторного регистра из набора, присвоенного одному из предыдущих векторных аргументов. Например, для некоторой функции `void foo (vint64m1_t a, vint64m4_t b, vint64m2_t c)`, аргумент `a` будет располагаться на регистре `v8`, аргумент `b` на регистрах `v12-15`, а аргумент `c` на регистрах `v10-v11`.

4 Отладчик GDB

4.1 Основная информация и возможности

GDB (GNU Project Debugger) — это инструмент для поиска ошибок в программах, предоставляющий средства анализа и управления их выполнением. Он поддерживает множество языков программирования и позволяет запускать приложения под своим контролем для пошагового изучения их поведения.

Одна из основных возможностей отладчика это остановка исполнения программы в некоторой точке. GDB позволяет устанавливать специальные метки в программе – breakpoint-ы (точки останова), которая указывает, в каком месте следует остановить выполнение программы. Breakpoint может быть привязан как к строке в коде, так и к определенному адресу. Во втором случае исполнение программы будет остановлено перед исполнением инструкции, расположенной по указанному адресу.

После остановки программы, GDB позволяет выполнять множество различных операций, полезных для поиска ошибок в коде.

Например, GDB позволяет просматривать значение переменных и изменять их. Для низкоуровневой отладки существует аналогичная возможность отслеживания и редактирования значений определенных регистров.

Следующей важной возможностью GDB является просмотр информации о текущем стековом фрейме и стеке вызова функций.

Также, весьма полезной функцией является вызов отдельных функций программы.

4.2 Отладочная информация: DWARF

Отладочная информация – данные, которые компилятор вставляет в исполняемые файлы. Эти данные позволяют GDB понять, какие существуют переменные, функции, их местонахождение, как соотносятся между собой машинные инструкции и исходный код и т. д.

DWARF – это современный формат отладочной информации, который используется в компиляторах GCC и LLVM и поддерживается GDB.

Стандарт DWARF разработан так, чтобы быть применимым к множеству существующих языков, а также иметь возможности к расширению

для будущих языков.

Запись отладочной информации (Debugging Information Entry, DIE) – это базовая единица данных в формате DWARF. Записи отладочной информации хранятся в специальных секциях объектного файла:

- `.debug_info` – основная информация о структурах, функциях, переменных и т. д.
- `.debug_str` – хранит строки, используемые в других секциях
- `.debug_line` – соответствие между строками исходного кода и адресами
- `.debug_loclists` – описывает местоположение переменных в различные моменты исполнения программы (на регистрах, стеке или в памяти)
- `.debug_rnglist` – список диапазонов адресов, используется, когда адреса для записей отладочной информации не могут быть описаны непрерывным диапазоном
- `.debug_frame` – информация для восстановления стека вызовов
- `.debug_macro` – описывает препроцессорные директивы (макросы)

DWARF использует наборы таких записей, чтобы описать структуру программы. Каждая запись состоит из тега (tag), определяющего тип описываемой сущности (например функция, переменная или название типа), и атрибутов, задающих конкретные свойства.

Рассмотрим генерацию DWARF информации на конкретном примере. Здесь и далее для компиляции я буду использовать LLVM Clang 19.1.0 (если не указано иное) для архитектуры RISC-V, а для наглядного отображения DWARF информации – инструмент `llvm-dwarfdump`. Данный инструмент сканирует DWARF секции в объектных файлах и выводит содержимое в удобочитаемом виде.

Итак, возьмем простую функцию с Листинга 1 и посмотрим, какую DWARF информацию генерирует для нее компилятор.

```
int foo (char a, int b[3], float *c) { return 42; }
```

Листинг 1: Пример простейшей функции

Для данной функции будет сгенерирована следующая DWARF информация.

```

0x00000091:  DW_TAG_subprogram
                DW_AT_low_pc      (0x00000000000000654)
                DW_AT_high_pc     (0x00000000000000674)
                DW_AT_frame_base  (DW_OP_reg8 X8)
                DW_AT_name       ("foo")
                DW_AT_decl_file   ("/app/example.c")
                DW_AT_decl_line   (1)
                DW_AT_prototyped  (true)
                DW_AT_type        (0x000000d1 "int")
                DW_AT_external    (true)

0x000000a0:  DW_TAG_formal_parameter
                DW_AT_location     (DW_OP_fbreg -17)
                DW_AT_name        ("a")
                DW_AT_decl_file   ("/app/example.c")
                DW_AT_decl_line   (1)
                DW_AT_type        (0x000000d5 "char")

0x000000ab:  DW_TAG_formal_parameter
                DW_AT_location     (DW_OP_fbreg -32)
                DW_AT_name        ("b")
                DW_AT_decl_file   ("/app/example.c")
                DW_AT_decl_line   (1)
                DW_AT_type        (0x000000d9 "int *")

0x000000b6:  DW_TAG_formal_parameter
                DW_AT_location     (DW_OP_fbreg -40)
                DW_AT_name        ("c")
                DW_AT_decl_file   ("/app/example.c")
                DW_AT_decl_line   (1)
                DW_AT_type        (0x000000de "float *")

...

0x000000d1:  DW_TAG_base_type
                DW_AT_name        ("int")
                DW_AT_encoding    (DW_ATE_signed)
                DW_AT_byte_size   (0x04)

0x000000d5:  DW_TAG_base_type
                DW_AT_name        ("char")
                DW_AT_encoding    (DW_ATE_unsigned_char)
                DW_AT_byte_size   (0x01)

0x000000d9:  DW_TAG_pointer_type
                DW_AT_type        (0x000000d1 "int")

0x000000de:  DW_TAG_pointer_type
                DW_AT_type        (0x000000e3 "float")

0x000000e3:  DW_TAG_base_type

```

```
DW_AT_name    ("float")
DW_AT_encoding (DW_ATE_float)
DW_AT_byte_size (0x04)
```

Листинг 2: Часть сгенерированной DWARF-информации, соответствующая функции `foo`

На Листинге 2 можно увидеть тег `DW_TAG_subprogram`, указывающий на то, что данная запись описывает некоторую функцию. По тегу `DW_AT_name` мы можем определить, что это функция с именем `foo`.

Атрибуты `DW_AT_low_pc` и `DW_AT_high_pc` описывают диапазон адресов в машинном коде, в котором располагается данная функция.

Содержимое `DW_AT_frame_base` описывает значение базового адреса фрейма данной функции. Для данной функции адрес ее фрейма будет располагаться на регистре `x8`.

Для описания типа возвращаемого функцией значения, используется атрибут `DW_AT_type`.

4.3 Необходимая информация о внутреннем устройстве GDB

Рассмотрим поэтапно процесс вызова функции с помощью команды `call` в GDB.

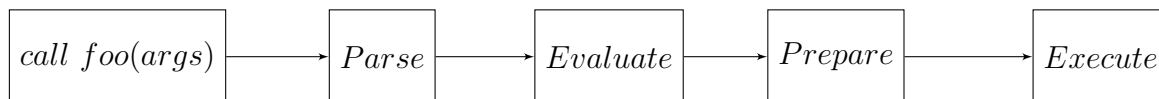


Рис. 2: Процесс исполнения команды `call`

После вызова пользователем команды `call foo(args)`, GDB передает полученное выражение на парсинг. На этапе парсинга происходит вычисление выражения `foo(args)`, строится дерево выражения. Далее, GDB начинает интерпретацию выражения путем обхода дерева. Когда GDB встречает узел, который соответствует вызову функции, он переходит к подготовке вызова. На этапе подготовки происходит выделение стекового фрейма, сохранение значений регистров, размещение аргументов функции в регистрах или памяти, установка PC (program counter) и SP (stack pointer). После этого GDB запускает исполнение программы до момента возврата из функции.

Далее приведена информация о некоторых функциях и структурах данных исходного кода GDB. Это позволит продемонстрировать, каким

образом была достигнута поддержка новых возможностей, а также объяснить, в какие части кода производились вмешательства.

- **type** – структура, описывающая конкретный экземпляр типа. Некоторые методы данной структуры:

- **code** – код данного типа. Например, `TYPE_CODE_INT`, `TYPE_CODE_ARRAY` или `TYPE_CODE_TYPEDEF`.
- **name** – имя данного типа.
- **length** – размер типа в байтах.
- **target_type** – метод, возвращающий указатель на объект `struct type`, т. е. информацию о каком-то другом типе. Интерпретировать информацию об этом новом типе данных следует в зависимости от исходного типа. Например, если исходный тип это некоторый массив (`code` возвращает `TYPE_CODE_ARRAY`), то **target_type** – это тип элементов, из которых этот массив состоит. В другом случае, если исходный тип является синонимом типа (`code` возвращает `TYPE_CODE_TYPEDEF`), то **target_type** – это тот тип, синонимом которого он является.

- Структура `riscv_arg_info` – хранит информацию об аргументе функции: его значение, тип, расположение (на регистре/нескольких регистрах или в памяти), размер.

```
struct riscv_arg_info
{
    const gdb_byte *contents;
    int length;
    int align;

    struct type *type;

    struct location
    {
        enum location_type
        {
            in_reg,
            in_several_regs,
            on_stack,
        } loc_type;

        union
        {
```



```

        int regno;
        int offset;
    } loc_data;

    int c_length;
    int c_offset;
} argloc[2];

bool is_unnamed;
};

```

Листинг 3: Структура `riscv_arg_info`

- Структура `riscv_call_info` – содержит информацию о том, куда можно поместить аргументы функции, отслеживает свободные регистры и области в памяти.

```

struct riscv_call_info
{
    riscv_call_info (struct gdbarch *gdbarch)
        : int_regs (RISCV_A0_REGNUM, RISCV_A0_REGNUM + 7),
          float_regs (RISCV_FA0_REGNUM, RISCV_FA0_REGNUM + 7),
          vector_regs (RISCV_V0_REGNUM + 8, RISCV_V0_REGNUM + 23)
    {
        xlen = riscv_abi_xlen (gdbarch);
        flen = riscv_abi_flen (gdbarch);
        vlenb = riscv_abi_vlenb (gdbarch);

        /* Reduce the number of integer argument registers when using the
           embedded abi (i.e. rv32e). */
        if (riscv_abi_embedded (gdbarch))
            int_regs.last_regnum = RISCV_A0_REGNUM + 5;

        /* Disable use of floating point registers if needed. */
        if (!riscv_has_fp_abi (gdbarch))
            float_regs.next_regnum = float_regs.last_regnum + 1;

        /* Disable use of vector registers if needed. */
        if (!riscv_has_vector_regs (gdbarch))
            vector_regs.available_intervals.clear ();
    }

    /* Track the memory areas used for holding in-memory arguments to a
       call. */
    struct riscv_memory_offsets memory;

    /* Holds information about the next integer register to use for passing
       an argument. */
    struct riscv_arg_reg int_regs;

    /* Holds information about the next floating point register to use for
       passing an argument. */
    struct riscv_arg_reg float_regs;

```

```
/* Holds information about the next vector register to use for  
passing an argument. */  
struct riscv_vector_arg_reg vector_regs;  
  
/* The XLEN and FLEN are copied in to this structure for convenience, and  
are just the results of calling RISCV_ABI_XLEN and RISCV_ABI_FLEN. */  
int xlen;  
int flen;  
int vlenb;  
};
```

Листинг 4: Структура `riscv_call_info`

5 Описание практической работы

5.1 Исправление генерации отладочной информации в компиляторе LLVM

В процессе проведения различных экспериментов с генерацией отладочной информации, было обнаружено, что для кортежных типов, таких как `vint8m4x2_t`, компилятор LLVM генерирует некорректную отладочную информацию о размере таких типов. Компилятор не учитывал множитель `NFIELDS`.

Ниже приведена часть исходного кода LLVM с исправлением этой ошибки:

```
bool Fractional = false;
unsigned LMUL;
unsigned NFIELDS = Info.NumVectors;
unsigned FixedSize = ElementCount * SEW;
if (Info.ElementType == CGM.getContext().BoolTy) {
    // Mask type only occupies one vector register.
    LMUL = 1;
} else if (FixedSize < 64) {
    // In RVV scalable vector types, we encode 64 bits in the fixed part.
    Fractional = true;
    LMUL = 64 / FixedSize;
} else {
    LMUL = FixedSize / 64;
}

// Element count = (VLENB / SEW) x LMUL x NFIELDS
SmallVector<uint64_t, 12> Expr(
    // The DW_OP_bregx operation has two operands: a register which is
    // specified by an unsigned LEB128 number, followed by a signed LEB128
    // offset.
    {llvm::dwarf::DW_OP_bregx, // Read the contents of a register.
    4096 + 0xC22,             // RISC-V VLENB CSR register.
    0, // Offset for DW_OP_bregx. It is dummy here.
    llvm::dwarf::DW_OP_constu,
    SEW / 8, // SEW is in bits.
    llvm::dwarf::DW_OP_div, llvm::dwarf::DW_OP_constu, LMUL});
if (Fractional)
    Expr.push_back(llvm::dwarf::DW_OP_div);
else
    Expr.push_back(llvm::dwarf::DW_OP_mul);
// NFIELDS multiplier
if (NFIELDS > 1)
    Expr.append({llvm::dwarf::DW_OP_constu, NFIELDS,
                 llvm::dwarf::DW_OP_mul});
// Element max index = count - 1
Expr.append({llvm::dwarf::DW_OP_constu, 1,
```

```
llvm::dwarf::DW_OP_minus});
```

Листинг 5: Исправление генерации информации о размере некоторых типов

Соответствующий патч с добавлением необходимого множителя `NFIELDS` был мною предложен и принят в исходный код компилятора LLVM.

5.2 Реализация в GDB

Для того, чтобы поддержать RISC-V Vector Calling Convention в отладчике GDB, необходимо было реализовать корректную передачу аргументов с векторными типами в функцию перед ее вызовом. Вторым требованием является сохранение значений определенных регистров и их восстановление после окончания исполнения функции, однако GDB самостоятельно сохраняет значения всех регистров перед вызовом и восстанавливает их после, поэтому реализовывать данное требование не потребовалось.

Процесс размещения аргументов функции можно условно разделить на две стадии: определение корректного местоположения аргументов функции в соответствии с ABI и размещение этих аргументов. В рамках решения поставленных задач был модифицирован процесс установление правильного размещения аргументов функции, добавлена поддержка определения правильного местоположения для векторных аргументов.

Итак, в первую очередь, для определения векторного типа по его имени была написана следующая простая функция:

```
static bool
is_rvv_type (struct type *type)
{
    if (type)
    {
        if (type->name () &&
            (strncmp (type->name (), "__rvv", 5) == 0))
            return true;
        else if (type->code () == TYPE_CODE_TYPEDEF)
            return is_rvv_type (type->target_type ());
    }

    return false;
}
```

Листинг 6: Функция для определения векторных типов

Например, тип `__rvv_int8m1_t` (`vint8m1_t`).

Для выбора способа передачи переменной векторного типа (через регистры или через память) была написана функция:

```
static void
riscv_call_arg_vector (struct riscv_arg_info *ainfo,
                      struct riscv_call_info *cinfo,
                      struct type *func_arg_type)
{
    gdb_assert (((ainfo->length % cinfo->vlenb) == 0)
               || ainfo->length < cinfo->vlenb);

    if (!riscv_assign_vec_reg_location (ainfo, &cinfo->vector_regs,
                                       func_arg_type, cinfo->vlenb))
    {
        // Try to pass value by reference
        ainfo->argloc[0].loc_type = riscv_arg_info::location::by_ref;
        cinfo->memory.ref_offset = align_up (cinfo->memory.ref_offset,
                                           ainfo->align);
        ainfo->argloc[0].loc_data.offset = cinfo->memory.ref_offset;
        cinfo->memory.ref_offset += ainfo->length;
        ainfo->argloc[0].c_length = ainfo->length;

        if (!riscv_assign_reg_location (&ainfo->argloc[1],
                                       &cinfo->int_regs,
                                       cinfo->xlen, 0))
            riscv_assign_stack_location (&ainfo->argloc[1],
                                       &cinfo->memory,
                                       cinfo->xlen, cinfo->xlen);
    }
}
```

Листинг 7: Функция для определения способа передачи значения аргумента

В первую очередь пытаемся разместить значение на векторных регистрах (в соответствии с RISC-V Vector Calling Convention) с помощью функции `riscv_assign_vec_reg_location` (подробное описание функции будет ниже). В случае неудачи, пытаемся передать значение по адресу, который, в свою очередь, пытаемся сначала разместить на регистре общего назначения, используя `riscv_assign_reg_location`, или, на стеке, с помощью `riscv_assign_stack_location`.

Реализация функции `riscv_assign_vec_reg_location`:

```
static bool
riscv_assign_vec_reg_location (struct riscv_arg_info *ainfo,
                              struct riscv_vector_arg_reg *reg,
                              struct type *func_arg_type, int vlenb)
{
    /* Amount of data that is stored on one register */
    int data_length = vlenb;
```

```

struct riscv_arg_info::location *loc0 = &ainfo->argloc[0];
struct riscv_arg_info::location *loc1 = &ainfo->argloc[1];

const char *func_arg_type_name
    = (func_arg_type) ? func_arg_type->name () : nullptr;

/* Try to get info about LMUL*/
const char *lmul_pos = (func_arg_type_name) ?
    strstr (func_arg_type_name, "m") : nullptr;
bool is_fractional_lmul = (lmul_pos &&
    strncmp (lmul_pos, "mf", 2) == 0) ?
    true : false;
int lmul = (lmul_pos) ?
    ((is_fractional_lmul) ?
    lmul_pos[2] - '0' : lmul_pos[1] - '0') : 1;

/* If it is a segment type, it's name should ends with 'xN_t',
   where N is * NFIELDS*/
const char *nfields_pos = (func_arg_type_name) ?
    strstr (func_arg_type_name, "x") : nullptr;
int nfields = (nfields_pos) ? nfields_pos[1] - '0' : 1;

int num_required_regs = (ainfo->length <= vlenb) ?
    1 : ainfo->length / vlenb;

if (is_fractional_lmul)
{
    num_required_regs = nfields;
    data_length = data_length / lmul;
}

int first_regnum = -1;

/* Representation of vector segmented types, like vint32m4x2_t,
   is different in Clang and GCC.
   In Clang, all types are arrays.
   In GCC, segmented types are struct's,
   other vector types are arrays. */
if (ainfo->type->code () == TYPE_CODE_ARRAY)
{
    if (ainfo->type->target_type ()->code () == TYPE_CODE_BOOL)
        first_regnum = reg->try_use_v0 ();
    else
        first_regnum = reg->get_interval_start (num_required_regs,
            nfields);
}
else if (ainfo->type->code () == TYPE_CODE_STRUCT)
{
    first_regnum = reg->get_interval_start (num_required_regs,
        nfields);
}

```

```

int last_regnum = first_regnum + num_required_regs - 1;

if (first_regnum != -1)
{
    loc0->loc_type = riscv_arg_info::location::in_several_regs;
    loc0->loc_data.regno = first_regnum;
    loc0->c_length = data_length;
    loc0->c_offset = 0;

    loc1->loc_type = riscv_arg_info::location::in_several_regs;
    loc1->loc_data.regno = last_regnum;
    loc1->c_length = 0;
    loc1->c_offset = 0;

    return true;
}

return false;
}

```

Листинг 8: Функция для выбора набора регистров для передачи аргумента

В данной функции мы сначала определяем параметры LMUL и NFIELDS. К сожалению, эти параметры пока невозможно получить более «чистым» способом – из отладочной информации, т. к. их там попросту нет. Мы обладаем только информацией о размере типа, из которой мы можем сделать вывод о количестве необходимых векторных регистров, но не о множителе LMUL, необходимом для корректной реализации векторного соглашения о вызовах. Далее, вызывая метод `get_interval_start` у объекта `struct riscv_vector_arg_reg *reg`, который сохраняет в себе информацию об уже занятых регистрах, мы получаем номер первого регистра в наборе (и вычисляем номер последнего регистра), в котором будет сохранено необходимое значение.

Как было сказано выше, для сохранения информации об уже занятых регистрах, была написана структура данных `riscv_vector_arg_reg`:

```

struct riscv_vector_arg_reg_interval
{
    int first;
    int last;
};

struct riscv_vector_arg_reg
{
    riscv_vector_arg_reg (int first, int last)
        : available_intervals{ { first, last } }
}

```

```

{
    /* Nothing. */
}

int
get_interval_start (int count, int NFIELDS = 1)
{
    gdb_assert (count % NFIELDS == 0);

    int LMUL = count / NFIELDS;
    for (auto cur_interval = available_intervals.begin ();
         cur_interval != available_intervals.end (); ++cur_interval)
    {
        int cur_first = cur_interval->first;
        int cur_last = cur_interval->last;
        if ((cur_first - RISC_V_VO_REGNUM) % LMUL != 0)
            /* According to RISC-V Vector ABI,
               first register number should be
               a multiple of LMUL */
        {
            cur_first -= (cur_first - RISC_V_VO_REGNUM) % LMUL;
            cur_first += LMUL;
        }

        if ((cur_last - cur_first + 1) >= count)
        {
            if (cur_first > cur_interval->first)
            {
                available_intervals.insert
                    (cur_interval, { cur_interval->first, cur_first - 1 });
            }

            cur_interval->first = cur_first + count;

            return cur_first;
        }
    }

    return -1;
}

int
try_use_v0 ()
{
    if (is_v0_used)
        return get_interval_start (1);
    else
    {
        is_v0_used = true;
        return RISC_V_VO_REGNUM;
    }
}

```



```

bool is_v0_used
    = false; /* To pass masks we should use v0, if it's available */

std::list<riscv_vector_arg_reg_interval> available_intervals;
};

```

Листинг 9: Структура для хранения информации о свободных векторных регистрах

Данные о незанятых векторных регистрах хранятся в виде интервалов вида `{first, last}` с номерами регистров. В случае, если нам необходимо выделить какой-то набор регистров, лежащий в интервале, этот интервал разбивается на 3 части: необходимый нам набор, интервал до и интервал после необходимого набора. Эти новые интервалы сохраняются взамен старого интервала (в случае, если они не являются пустыми).

Кроме того, отдельно отслеживается занятость регистра `v0`, т. к. он используется для передачи векторных масок.

Последняя важная часть, это обработка введенного варианта расположения переменной на регистре `riscv_arg_info::location::in_several_regs` в функции `riscv_push_dummy_call`:

```

case riscv_arg_info::location::in_several_regs:
{
    const gdb_byte *cur_contents = info->contents;
    int cur_c_length = info->argloc[0].c_length;
    for (int cur_regno = info->argloc[0].loc_data.regno;
        cur_regno <= info->argloc[1].loc_data.regno; cur_regno++)
    {
        riscv_regcache_cooked_write (cur_regno, cur_contents,
                                     cur_c_length, regcache,
                                     call_info.flen);
        cur_contents += cur_c_length;
    }

    second_arg_length = 0;
}

```

Листинг 10: Запись необходимого значения в соответствующие регистры в кэше

6 Заключение

В рамках данной работы были решены все поставленные задачи:

- Изучена генерация отладочной информации для архитектуры RISC-V компиляторами GCC и LLVM

В рамках этой задачи была подробно рассмотрена архитектура RISC-V, ее особенности и преимущества. Также, было подробно изучено векторное расширение данной архитектуры и специфика соглашения о вызовах для векторных типов.

- Реализовано векторное соглашение о вызовах в отладчике GDB
- Поддержано определение векторных типов в отладчике GDB

Была выполнена основная цель работы: реализовать в отладчике GDB для архитектуры RISC-V поддержку вызова функций, использующих векторный ABI.

Корректность работы данной реализации была проверена на тестовых сценариях в рамках тестовой инфраструктуры GDB.

Патч с поддержкой функциональности планируется к включению в Syntacore Development Toolkit (SC-DT) и публикации в открытом доступе.

В рамках дальнейшей работы планируется расширить спектр поддерживаемых функциональностей в отладчике GDB для векторного расширения RISC-V. Одним из направлений может стать реализация обратного исполнения с учетом специфики векторного расширения RISC-V.

Список литературы

- [1] The RISC-V Instruction Set Manual Volume I Unprivileged, 2025. — <https://drive.google.com/file/d/1uviu1nH-tScFfgrovvFCrj70mv8tFtkp>.
- [2] The RISC-V Instruction Set Manual Volume II: Privileged, 2025. — https://drive.google.com/file/d/17GeetSnT5wW3xNuAHI95-SI1gPGd5sJ_.
- [3] RISC-V ABIs Specification, Document Version 1.0, 2022. — https://drive.google.com/file/d/1Ja_Tpp_5Me583CGVD-BIZMlgGBnlKU4R.
- [4] RISC-V Vector C Intrinsic Specification, 2025. — <https://drive.google.com/file/d/1RTZi2iOLKzqaX95JCCnzw0m7iCIN3JEq>.
- [5] DWARF Debugging Information Format, Version 5, 2017. — <https://dwarfstd.org/doc/DWARF5.pdf>.
- [6] Debugging with GDB, 2025. — <https://sourceware.org/gdb/current/onlinedocs/gdb.pdf>.